NOVA: An Explainable, Workload-Aware Compiler Optimization Framework for Numerical Programs

Two parts: **NOVA Compiler** (educational compiler for NOVA Script) and **NOVA SmartPass** (research track: tuning GCC flags for numerical C++ kernels)

Course: Compiler Design

Problem statement

1. Which flags?

- `-O2` and `-O3` are only two of many GCC optimization flags.
- Their effect depends on the program; for loop unrolling the manual says it “may or may not make it run faster” [1].
- Every candidate costs one compile and one run to test.

2. Still correct?

- A faster binary is useless if its answer changes.
- `-Ofast` enables transformations “not valid for all standard-compliant programs” [1].
- So each candidate must be checked against a reference.

3. Speedup real?

- Run times vary: on our machine the baseline spread (IQR) reached about 7% pinned and about 25% unpinned.
- So we need repeated runs, medians and saved raw samples [3, 4].
- A choice tuned on one input size may not hold on others.

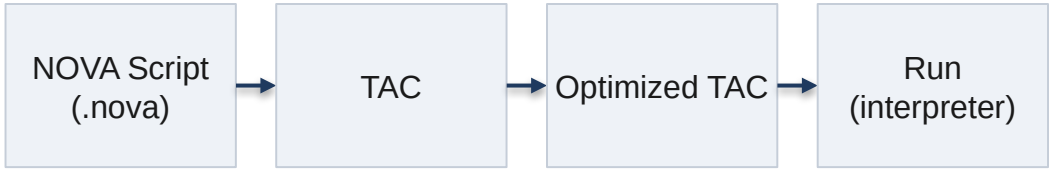
Scope of NOVA: three small numerical C++ kernels (matmul, vector_add, stencil) and GCC flags only. It does not claim that every developer has this problem; no survey was done.

Proposed solution: two parts of NOVA

nova (one command-line tool, C++17)

NOVA Compiler

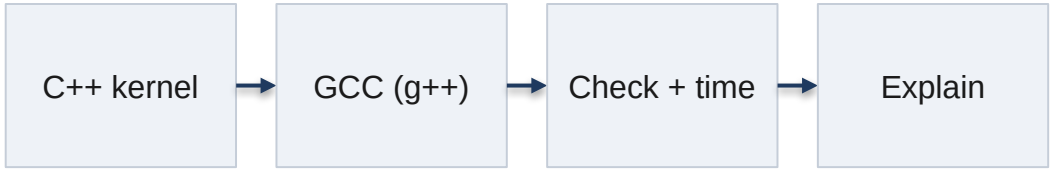
educational track



Shows how a compiler works inside: tokens, syntax tree, simple intermediate code, and four safe optimizations. Runs programs on reference interpreters.

NOVA SmartPass

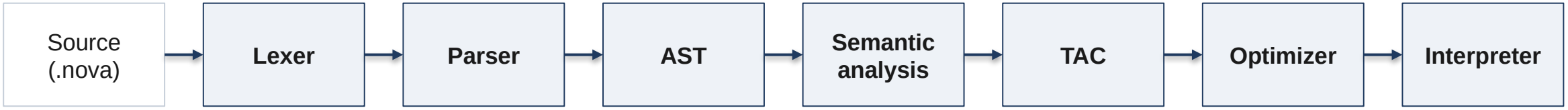
research track



Asks GCC to build a numerical kernel with chosen flags, checks the output, measures, and records why a configuration won. **GCC does all the compiling.**

Objectives: (A) show every compiler stage working on a small language; (B) test a bounded, workload-aware, correctness-checked search over GCC flags and keep the evidence. The two parts are kept separate on purpose.

NOVA Compiler: pipeline



All stages shown are implemented. No machine-code backend: programs run on the AST and TAC interpreters.

Stage	What it does and why it matters
Lexer	Splits text into tokens (keyword, identifier, integer, operator) with line:column for error messages.
Parser	Recursive descent, one function per precedence level; reports the first syntax error with its location.
AST	Tree of the program (Let, Print, Binary…), independent of how the text was laid out.
Semantic analysis	Symbol table: rejects use of an undeclared variable and duplicate declarations.
TAC	Each instruction has at most one operator, e.g. `$t1 = x * 2`: simple enough to analyze and optimize.
Optimizer	Four passes repeated until nothing changes (a fixed point); see slide 10.
Interpreter	Runs the AST or the TAC; division by zero is a runtime error with a location.

NOVA Script: integers only (64-bit wrapping), `let`, assignment, `print`, `+ - * /`, parentheses; no loops or branches yet.

How NOVA SmartPass Chooses Compiler Flags

1 Examine the workload → **2 Select a candidate** → **3 Compile with GCC** → **4 Check correctness** → **5 Measure** → **6 Record evidence**

Repeat until the search budget is used (demonstration setting: at most 4 searched candidates).

Five signals used to choose the next candidate, and their real status

1 Source-code features

Per-kernel facts (size n, working set, streaming?, reduction?) registered for matmul, vector_add, stencil. Not parsed from arbitrary C++.

Used

2 Loop depth

How deeply the kernel's loops nest (recorded: matmul 3, stencil 2). Deeper does not mean faster.

Recorded, not scored

3 Arithmetic intensity

FLOPs ÷ bytes moved, estimated from the kernel definition and n.

Recorded, not scored

4 Memory-bandwidth demand

Proxy only: working set above 4 MiB favors prefetch flags. No bandwidth model, no hardware counters.

Proxy used

5 Historical evidence

Earlier log-gains vs -O2; closer input sizes count more. Needs a history file.

Used if file given

Teaching example for signal 2: two nested loop levels

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
    result[i] += a[i][j] * x[j];
  }
}
```

Score used today = prior (streaming, reduction, working set > 4 MiB) + history + pilot feedback. A hand-written heuristic, not machine learning.

Loop depth and FLOPs/byte are recorded for explanation; they do not change the order yet.

Candidates, Budget and Accounting

Illustrative candidates for B = 4

	Candidate (flag ID)	Why test it?	Status
A	`O2+prefetch+native`	Conservative: stays at -O2, adds prefetch	Illustrative
B	`O3+unroll+native`	Stronger optimization (-O3, unrolling)	Illustrative
C	`O2+prefetch+vec+native`	Different supported flag combination	Illustrative
D	`O3+prefetch+unroll`	Chosen from workload features or history	Illustrative

Real IDs from the 22-candidate space (`-march=native` appears as `native`). A to D are not a measured adaptive run and not a claim of a global optimum.

What uses the budget? (B = 4)

Event	Uses budget?
Invalid flag set	No: refused before any build
Baselines -O2, -O3	No: always built
Build fails	Yes: still counted
Wrong answer	Yes: counted, cannot win
Valid measurement	Yes
Retry of a failure	None exist

Selected
4 candidates (the budget)

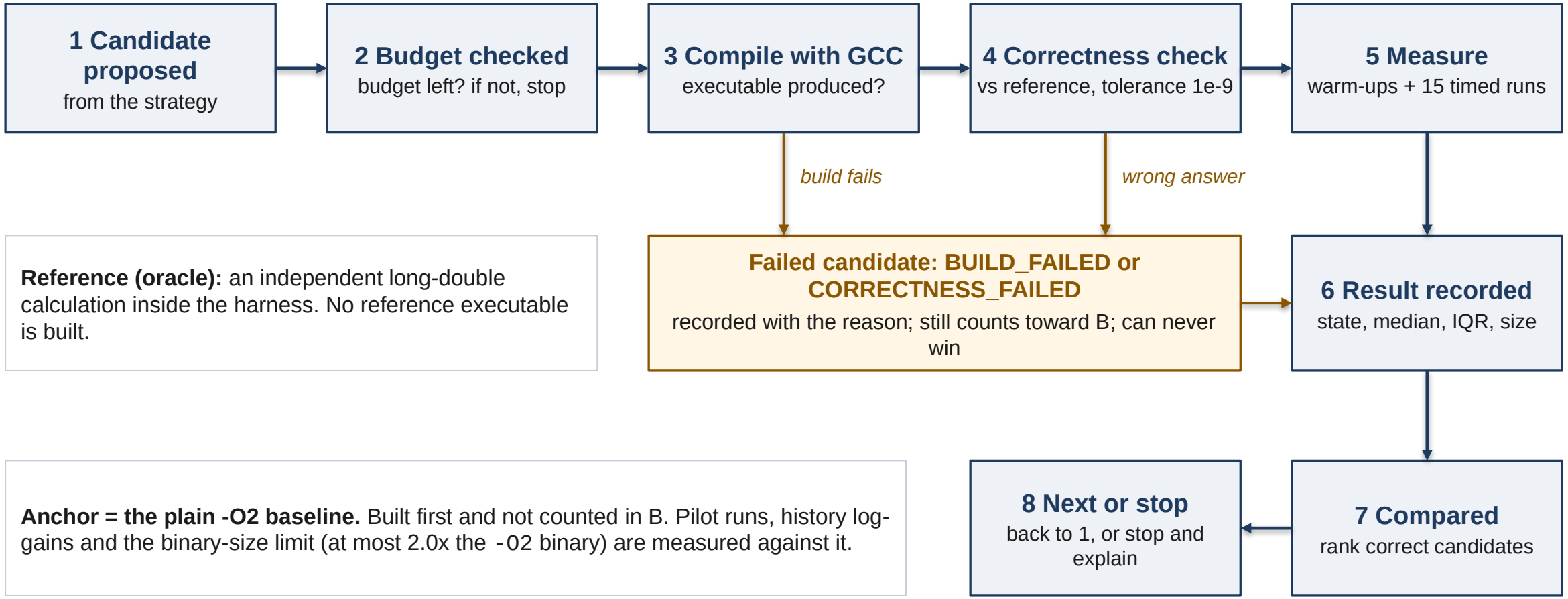
→

Compiled
4 attempts + 2 baselines = 6 builds

→

Valid measurements
0 to 4 searched, plus baselines

Candidate Lifecycle



Reference (oracle): an independent long-double calculation inside the harness. No reference executable is built.

Anchor = the plain -O2 baseline. Built first and not counted in B. Pilot runs, history log-gains and the binary-size limit (at most 2.0x the `-O2` binary) are measured against it.

In the code, candidates are built one by one first (adaptive: with a short pilot against the anchor); validation and timing then run for all built candidates together, interleaved. States: BUILD_FAILED, CORRECTNESS_FAILED, REJECTED_BY_POLICY, ELIGIBLE.

How NOVA Checks That Optimization Has Not Changed the Answer

Five numerical checks (each kernel prints them)

#	Check	Value compared
1	Total scalar sum	`sum` = v[0] + … + v[len−1]
2	First element	`s0` = v[0]
3	Middle element	`s1` = v[len / 2]
4	Final element	`s2` = v[len − 1]
5	Weighted-index checksum	`wsum` = Σ ((i mod 7) + 1) · v[i]

Rule (tolerance 1e-9)

`|actual − expected| ≤ 1e-9 · max(1, |expected|)`

Absolute near zero, relative for large values.

Kernel, n, seed and output length must also match. NaN or infinity fails. No separate empty-output rule: the length must equal the expected length.

Worked example (illustrative values)

Check	Expected	Candidate	|diff|	Allowed	Result
`sum`	1048.5732	1048.5732000004	4.0e-10	1.05e-6	Pass
`s1`	0.5	0.5000000021	2.1e-9	1.0e-9	Fail

One failed check rejects the whole candidate: it is recorded as CORRECTNESS_FAILED and cannot win, even if it is the fastest.

A gate, not a proof

- Five statistics can match while individual elements differ; the project does not compare every element.
- Where practical, a full comparison would be stronger; a scalar-only program would need its own output checks.
- The expected values come from a long-double reference inside the harness, not from a GCC-built executable.

Faster Code vs. Strict Floating-Point Behavior

What -ffast-math relaxes

- **Reassociation:** may regroup `(a + b) + c` as `a + (b + c)`; the rounding differs.
- **NaN / infinity:** may assume values are never NaN or infinite, so checks like `x != x` can disappear.
- **Signed zero:** may treat `-0.0` and `+0.0` as the same value.
- **Reproducibility:** the last digits can depend on compiler version, flags and target.

Operation-order example (double precision)

```
a = 1e16;  b = -1e16;  c = 1.0;
(a + b) + c   ->  1.0
a + (b + c)   ->  0.0   // b + c rounds to -1e16
```

Why strict rules matter in some fields

- **Finance:** audited rounding and the same figure on every re-run.
- **Aerospace / space:** specified, verifiable numerics; NaN and overflow must be detected, not assumed away.
- This does **not** mean every use of `-ffast-math` is unsafe; many simulation and graphics programs accept small differences.

In NOVA

- `-ffast-math` is not in the SmartPass whitelist; such a candidate is refused.
- `-Ofast` appears only in a separate disclosure run (V0.6) and passed the five checks on all three kernels.
- So the gate cannot show a result is safe for a high-stakes program. Use of fast-math there needs domain-specific validation.

Running NOVA on a Program (A): NOVA Compiler

1. Build once

```
cmake -S . -B build \
      -G "MinGW Makefiles"
cmake --build build
```

2. A NOVA Script program (not C++)

examples/basic.nova

```
let x = 10;
let y = 20;
let z = x * 2 + y;
print z;
```

NOVA Script has integers, `let`, `print` and `+ - * /` only. GCC and C++ are not involved in this part.

3. Compile to TAC, then run

```
$ build/nova tac examples/basic.nova
x = 10
y = 20
$t1 = x * 2
z = $t1 + y
print z

$ build/nova run examples/basic.nova
40
```

TAC = three-address code: at most one operator per line; `$t1` is a compiler temporary.

4. Optimize

```
// examples/optimize_demo.nova
let a = 4;
let b = a * 2 + 0;
let unused = b - b;
let z = 1 - 1;
print b * 1;
print a * (3 + 4);
```

```
$ build/nova opt \
    examples/optimize_demo.nova \
    --show-all
... (every pass)
== final (fixed point after
   3 iterations) ==
print 8
print 28
```

Four passes (fold, propagate, simplify, dce) repeat to a fixed point. Output is unchanged: 8 and 28.

Output copied from docs/DEMO.md (scripts/make_demo.sh, 2026-10-14). `--show-all` prints every pass. No machine-code backend: programs run on interpreters.

Running NOVA on a Program (B): SmartPass

Kernel (C++): benchmarks/kernels/stencil.cpp

```
for (std::size_t i = 1; i + 1 < n; ++i)
  next[i] = 0.25 * cur[i-1] + 0.5 * cur[i] + 0.25 * cur[i+1];
```

1 Baseline: GCC alone

```
g++ -std=c++17 -msse2 -mfpmath=sse -O2 …
(same with -O3; built in every run)
```

2 Reference

```
built-in long-double oracle; no reference executable
```

3 Tune (budget = 4)

```
nova tune stencil --budget=4 --strategy=adaptive \
     --history=FILE --record-history
```

4 Explain

```
nova explain benchmarks/results/<experiment-id>
```

Recorded in `benchmarks/results/<id>/`: manifest.json, results.json/csv, raw_timings.csv, logs/. `--record-history` also appends to the history file.

Comparison: saved run, stencil n = 200,000

Configuration	Median (ms)	IQR	Binary (B)	Correct
GCC `-O2`	16.178	0.550	81,660	Pass
GCC `-O3`	11.677	0.233	81,746	Pass
Selected: `O3+unroll+native`	8.869	0.248	84,306	Pass

- Same source, compiler, machine and input; 15 timed rounds, median and IQR; raw samples are saved.
- Selected vs better baseline (`-O3`): ratio of medians **1.317**; IQRs do not overlap.
- Tuning overhead: **11.49 s** for 6 builds plus timing.
- This saved run used the **random** strategy (seed 1, B = 4). No adaptive B = 4 run is saved, so the adaptive command is the supported syntax, not a result.
- Tuning size only: not evidence for other sizes.

SmartPass is not a compiler: GCC builds every binary. Real output: state/evidence/V0.7/20_explain_V0.7-stencil-random-s1-B4.log.

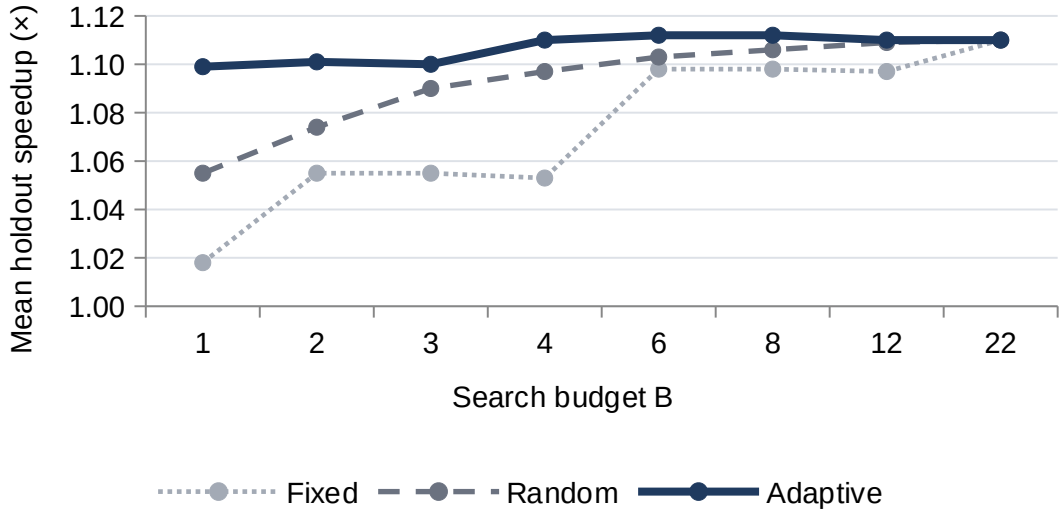
What NOVA Adds

Proposed contribution. NOVA SmartPass combines workload-feature-guided candidate selection, a strict four-candidate experimental budget, numerical correctness gating, and recorded evidence to make compiler-configuration tuning more controlled and explainable.

Why each piece helps

- **Budget:** bounds the experiment; failed builds still count.
- **Workload features:** decide which candidates are tried first.
- **History:** helps when a history file is supplied (implemented, optional).
- **Numerical validation:** rejects results whose answer changed.
- **Recorded evidence:** explains why a candidate was accepted or rejected (`nova explain`).

Demonstrated so far (replay study)



- At B = 4: adaptive 1.110 vs random 1.097: a small gap that shrinks as B grows (tie at B = 22).
- One machine, GCC 6.3.0, three kernels. Individual signals: inconclusive.

Source: report/report.md §8.2 and §9.

Current progress

Component	Status	Evidence
Lexer, parser, AST, semantic checks, TAC	**Complete**	Phases V0.2 to V0.4; located diagnostics; AST and TAC interpreters agree
TAC optimizer (4 passes)	**Complete**	V0.5; differential test on 4000 random programs
Benchmark lab, -O2/-O3 baselines, oracle	**Complete**	V0.6; 3 kernels; raw samples and manifests saved
SmartPass fixed and random search, correctness gate	**Complete**	V0.7; 12 equal-budget runs saved
Adaptive search, ablations, holdout evaluation	**Complete**	V0.8; 7260 replayed runs, 27 real runs
Evaluation report	**Complete**	report/report.md generated from saved data (V0.9)
Whole test suite	**Complete**	49/49 CTest pass in the V1.0 log
Linux (POSIX) process runner	**Partial**	Written, never compiled or run
Sanitizers (ASan, UBSan)	**Not run**	Not available for this GCC; checked containers used instead
Newer GCC, other machines	**Not done**	Outside the verified configuration

Release status: V1.0 RELEASED for the verified configuration only (Windows 11, MinGW.org GCC 6.3.0). See state/ROADMAP_STATUS.md and docs/RELEASE.md.

Limitations, Future Scope and Conclusion

Limitations (documented)

- One machine, old 32-bit GCC 6.3.0; three small kernels.
- Only 22 searchable configurations.
- Timing noise: baseline spread up to 7.2% (median 2.1%).
- Correctness gate: five statistics, not every element.
- Loop depth and arithmetic intensity are recorded but not yet scored; no bandwidth model.
- Adaptive search costs extra tuning time (pilot runs).

Future scope (not implemented)

- Linux and newer GCC; compile and test the POSIX runner.
- Sanitizer runs on a toolchain that supports them.
- More kernels, larger inputs, a bigger flag space.
- Use loop depth and arithmetic intensity in scoring.
- Compare every output element where practical.

Conclusion. NOVA keeps two questions apart: how a compiler works (NOVA Compiler) and how to compare GCC settings carefully (SmartPass). In the measured setting, adaptive search was slightly ahead of random at small budgets; this does not show NOVA is faster than GCC or that results generalize.

References: [1] GNU Project, GCC manual, “Options That Control Optimization” [2] Ansel et al., OpenTuner, PACT 2014 [3] Google Benchmark, “Random Interleaving” [4] SPEC CPU 2026 Run and Reporting Rules [5] LLVM, “My First Language Frontend”. Links: NOVA_Slide_Content.md.

Glossary

Term	Meaning in NOVA
Workload	A kernel plus its input size, e.g. stencil with n = 200,000.
Candidate	One whitelisted flag combination, e.g. `O3+unroll+native`.
Flag	A single GCC option, e.g. `-funroll-loops`.
Binary	The executable GCC builds for one candidate.
Budget (B)	Max searched candidates built; the -O2/-O3 baselines are extra.
Adaptive search	Next candidate chosen by prior + history + pilot feedback.
Accounting rules	Selected and built = counted, even if it fails; no retries.

Term	Meaning in NOVA
Correctness gate	Five output statistics must match the reference or the candidate cannot win.
Tolerance	Allowed gap: `1e-9 · max(1, |expected|)`
Anchor	The plain `-O2` baseline: reference for pilot runs, history and binary size.
“Unaccounted anchor”	Not defined in the project. Closest: the uncounted -O2/-O3 anchor baselines. Author to confirm.
Arithmetic intensity	Floating-point operations per byte moved.
Tuning overhead	Wall-clock time of the whole search (11.49 s in the saved stencil run).
Reference	Independent long-double calculation of the expected statistics.

Definitions follow the code (src/tune/, src/bench/) and docs/smartpass.md; the budget is configurable, 4 is the demonstration setting.